Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2
з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
Розробка графічного редактора об’єктів

Виконала
студентка групи ІМ-о51
Глазкова Ілона Андріївна
Номер в списку - 1

Перевірив
Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки новий проект з ім’ям Lab2.

2. Скомпілювати проект і отримати виконуваний файл програми.

3. Перевірити роботу програми. За потреби налагодити програму.

4. Проаналізувати та прокоментувати результати та вихідний текст

програми.

5. Оформити звіт

Завдання згідно варіанту:
Ліміт елементів: 101
Тип масиву: Статичний
"Гумовий" слід: Суцільна лінія червоного кольору
Позначка поточного об'єкту: В заголовку вікна програми
Увід прямокутника: Від центру до одного з кутів
Відображення прямокутника: Чорний контур з світло-зеленим заповненням
Увід еліпсу: По двом протилежним кутам охоплюючого прямокутника
Відображення еліпс: Чорний контур з білим заповненням

Код

Shape.h

```#pragma once
#include <windows.h>

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
public:
    Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}
    virtual ~Shape() {}

    void Set(long x1, long y1, long x2, long y2) {
        xs1 = x1; ys1 = y1; xs2 = x2; ys2 = y2;
    }

    virtual void Draw(HDC hdc) = 0;
    virtual void DrawRubber(HDC hdc) = 0;
};

class PointShape : public Shape {
public:
    void Draw(HDC hdc) override;
    void DrawRubber(HDC hdc) override;
};
```

```
class LineShape : public Shape {
public:
    void Draw(HDC hdc) override;
    void DrawRubber(HDC hdc) override;
};

class RectShape : public Shape {
public:
    void Draw(HDC hdc) override;
    void DrawRubber(HDC hdc) override;
};

class EllipseShape : public Shape {
public:
    void Draw(HDC hdc) override;
    void DrawRubber(HDC hdc) override;
};
```

shape.cpp

```
```#include "shape.h"

void PointShape::Draw(HDC hdc) {
    SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));
}
void PointShape::DrawRubber(HDC hdc) {
    SetPixel(hdc, xs2, ys2, RGB(255, 0, 0));
}

void LineShape::Draw(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    MoveToEx(hdc, xs1, ys1, NULL);
    LineTo(hdc, xs2, ys2);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
}
void LineShape::DrawRubber(HDC hdc) {
    MoveToEx(hdc, xs1, ys1, NULL);
    LineTo(hdc, xs2, ys2);
}

void RectShape::Draw(HDC hdc) {
    int left = 2 * xs1 - xs2;
    int top = 2 * ys1 - ys2;
    int right = xs2;
    int bottom = ys2;

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HBRUSH hBrush = CreateSolidBrush(RGB(144, 238, 144));

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

    Rectangle(hdc, left, top, right, bottom);
```
```

```
    SelectObject(hdc, hOldPen);
    SelectObject(hdc, hOldBrush);
    DeleteObject(hPen);
    DeleteObject(hBrush);
}
void RectShape::DrawRubber(HDC hdc) {
    int left = 2 * xs1 - xs2;
    int top = 2 * ys1 - ys2;
    Rectangle(hdc, left, top, xs2, ys2);
}

void EllipseShape::Draw(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HBRUSH hBrush = (HBRUSH)GetStockObject(WHITE_BRUSH);

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

    Ellipse(hdc, xs1, ys1, xs2, ys2);

    SelectObject(hdc, hOldPen);
    SelectObject(hdc, hOldBrush);
    DeleteObject(hPen);
}
void EllipseShape::DrawRubber(HDC hdc) {
    Ellipse(hdc, xs1, ys1, xs2, ys2);
}
```

Lab2.cpp

```
```#include "framework.h"
#include "Lab2.h"
#include "shape.h"

#define MAX_LOADSTRING 100
#define MAX_SHAPES 101

// Global Variables:
Shape* pcshape[MAX_SHAPES];
int shapeCount = 0;
Shape* currentShape = nullptr;
HINSTANCE hInst;   // current instance
WCHAR szTitle[MAX_LOADSTRING];   // The title bar text
WCHAR szWindowClass[MAX_LOADSTRING];   // the main window class name

bool isDrawing = false;
int currentMode = 0;

// Forward declarations of functions included in this code module:
ATOM   MyRegisterClass(HINSTANCE hInstance);
BOOL   InitInstance(HINSTANCE, int);
LRESULT CALLBACK   WndProc(HWND, UINT, WPARAM, LPARAM);
INT_PTR CALLBACK   About(HWND, UINT, WPARAM, LPARAM);

int APIENTRY wWinMain(_In_ HINSTANCE hInstance,
```
```

```
                     _In_opt_ HINSTANCE hPrevInstance,
                     _In_ LPWSTR    lpCmdLine,
                     _In_ int       nCmdShow)
{
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);

    // TODO: Place code here.

    // Initialize global strings
    LoadStringW(hInstance, IDS_APP_TITLE, szTitle, MAX_LOADSTRING);
    LoadStringW(hInstance, IDC_LAB2, szWindowClass, MAX_LOADSTRING);
    MyRegisterClass(hInstance);

    // Perform application initialization:
    if (!InitInstance (hInstance, nCmdShow))
    {
        return FALSE;
    }

    HACCEL hAccelTable = LoadAccelerators(hInstance, MAKEINTRESOURCE(IDC_LAB2));

    MSG msg;

    // Main message loop:
    while (GetMessage(&msg, nullptr, 0, 0))
    {
        if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
        {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }

    return (int) msg.wParam;
}



//
//  FUNCTION: MyRegisterClass()
//
//  PURPOSE: Registers the window class.
//
ATOM MyRegisterClass(HINSTANCE hInstance)
{
    WNDCLASSEXW wcex;

    wcex.cbSize = sizeof(WNDCLASSEX);

    wcex.style          = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc    = WndProc;
    wcex.cbClsExtra     = 0;
    wcex.cbWndExtra     = 0;
    wcex.hInstance      = hInstance;
    wcex.hIcon          = LoadIcon(hInstance, MAKEINTRESOURCE(IDI_LAB2));
    wcex.hCursor        = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground  = (HBRUSH)(COLOR_WINDOW+1);
```

```
    wcex.lpszMenuName   = MAKEINTRESOURCEW(IDC_LAB2);
    wcex.lpszClassName  = szWindowClass;
    wcex.hIconSm        = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(IDI_SMALL));

    return RegisterClassExW(&wcex);
}

//
//   FUNCTION: InitInstance(HINSTANCE, int)
//
//   PURPOSE: Saves instance handle and creates main window
//
//   COMMENTS:
//
//        In this function, we save the instance handle in a global variable and
//        create and display the main program window.
//
BOOL InitInstance(HINSTANCE hInstance, int nCmdShow)
{
   hInst = hInstance; // Store instance handle in our global variable

   HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
      CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);

   if (!hWnd)
   {
      return FALSE;
   }

   ShowWindow(hWnd, nCmdShow);
   UpdateWindow(hWnd);

   return TRUE;
}

//
//  FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
//  PURPOSE: Processes messages for the main window.
//
//  WM_COMMAND  - process the application menu
//  WM_PAINT    - Paint the main window
//  WM_DESTROY  - post a quit message and return
//
//
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static long x1, y1, x2, y2;

    switch (message)
    {
    case WM_COMMAND:
        {
            int wmId = LOWORD(wParam);
            // Parse the menu selections:
            switch (wmId)
            {
            case IDM_SHAPE_POINT:
```

```
            currentMode = 0;
            SetWindowText(hWnd, L"Lab2 - Object : Dot");
            break;
        case IDM_SHAPE_LINE:
            currentMode = 1;
            SetWindowText(hWnd, L"Lab2 - Object : Line");
            break;
        case IDM_SHAPE_RECT:
            currentMode = 2;
            SetWindowText(hWnd, L"Lab2 - Object : Rectangle");
            break;
        case IDM_SHAPE_ELLIPSE:
            currentMode = 3;
            SetWindowText(hWnd, L"Lab2 - Object : Ellipse");
            break;
        case IDM_ABOUT:
            DialogBox(hInst, MAKEINTRESOURCE(IDD_ABOUTBOX), hWnd, About);
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;
        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
        }
    }
    break;


case WM_LBUTTONDOWN:
    if (shapeCount < MAX_SHAPES) {
        x1 = LOWORD(lParam);
        y1 = HIWORD(lParam);
        x2 = x1; y2 = y1;
        isDrawing = true;

        switch (currentMode) {
        case 0: currentShape = new PointShape(); break;
        case 1: currentShape = new LineShape(); break;
        case 2: currentShape = new RectShape(); break;
        case 3: currentShape = new EllipseShape(); break;
        }
        currentShape->Set(x1, y1, x2, y2);
    }
    break;

case WM_MOUSEMOVE:
    if (isDrawing && currentShape != nullptr) {
        HDC hdc = GetDC(hWnd);

        SetROP2(hdc, R2_NOTXORPEN);
        HPEN hPenRubber = CreatePen(PS_SOLID, 1, RGB(255, 0, 0));
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPenRubber);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH));

        currentShape->DrawRubber(hdc);

        x2 = LOWORD(lParam);
        y2 = HIWORD(lParam);
```

```
            currentShape->Set(x1, y1, x2, y2);

            currentShape->DrawRubber(hdc);

            SelectObject(hdc, hOldPen);
            SelectObject(hdc, hOldBrush);
            DeleteObject(hPenRubber);
            ReleaseDC(hWnd, hdc);
        }
        break;

    case WM_LBUTTONUP:
        if (isDrawing && currentShape != nullptr) {
            isDrawing = false;

            x2 = LOWORD(lParam);
            y2 = HIWORD(lParam);
            currentShape->Set(x1, y1, x2, y2);
            pcshape[shapeCount] = currentShape;
            shapeCount++;

            currentShape = nullptr;
            InvalidateRect(hWnd, NULL, TRUE);
        }
        break;

    case WM_PAINT:
    {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);

        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i]) {
                pcshape[i]->Draw(hdc);
            }
        }

        EndPaint(hWnd, &ps);
    }
    break;

    case WM_DESTROY:
        for (int i = 0; i < shapeCount; i++) {
            delete pcshape[i];
        }
        PostQuitMessage(0);
        break;

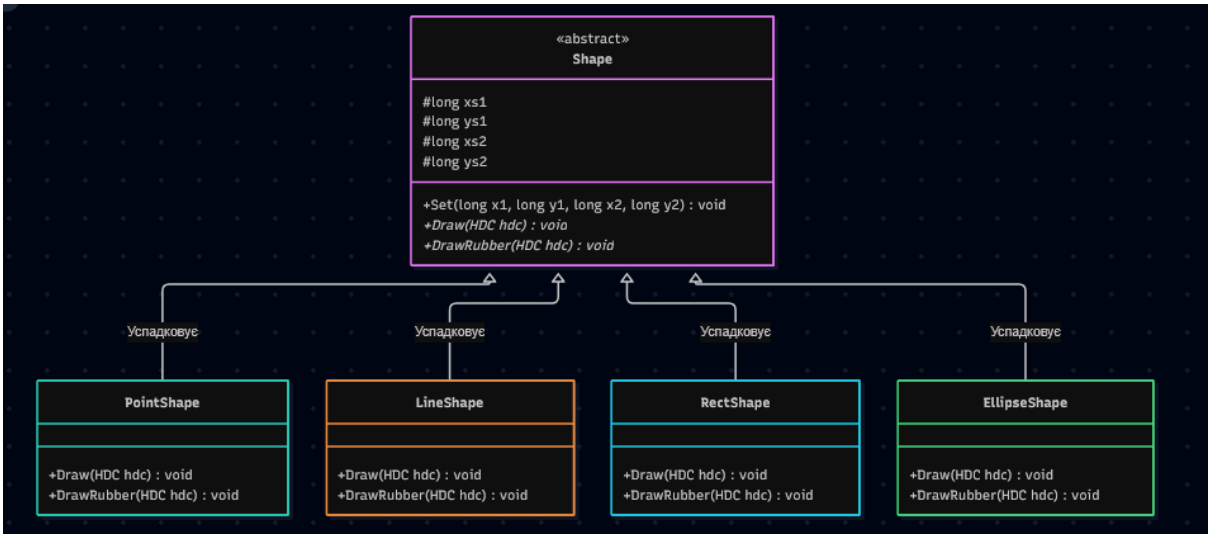
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

// Message handler for about box.
INT_PTR CALLBACK About(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    UNREFERENCED_PARAMETER(lParam);
```

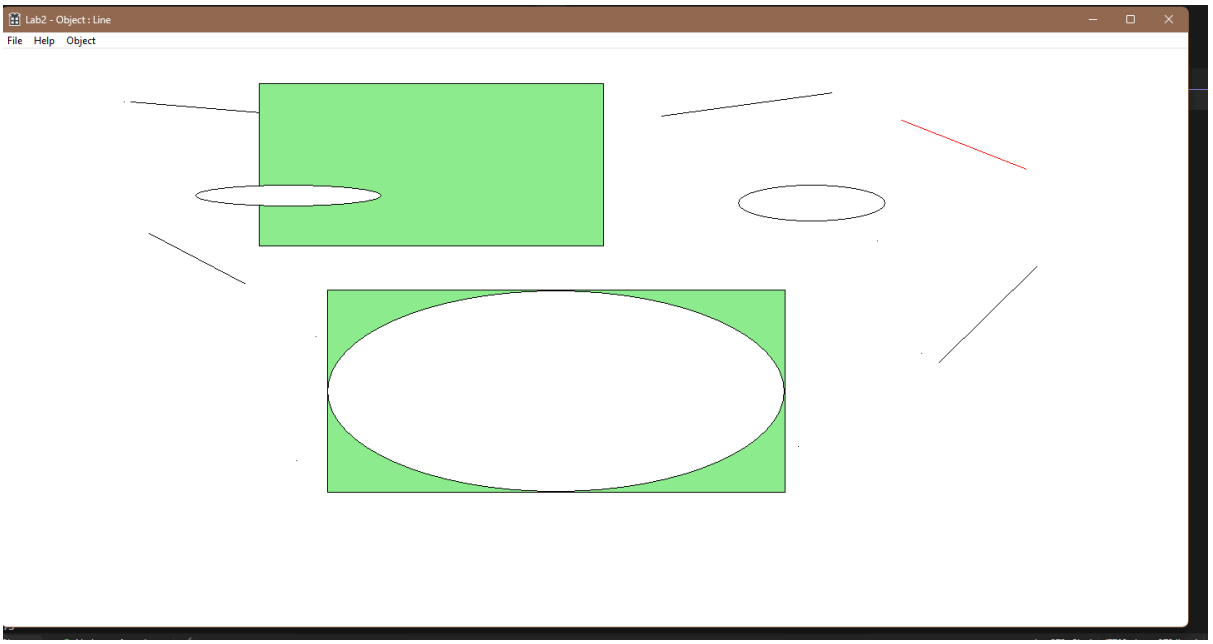
```
    switch (message)
    {
    case WM_INITDIALOG:
        return (INT_PTR)TRUE;

    case WM_COMMAND:
        if (LOWORD(wParam) == IDOK || LOWORD(wParam) == IDCANCEL)
        {
            EndDialog(hDlg, LOWORD(wParam));
            return (INT_PTR)TRUE;
        }
        break;
    }
    return (INT_PTR)FALSE;
}
```

Ієрархія класів



Скріншоти



Висновок

Під час створення графічного редактора я на практиці закріпила принципи ООП, використавши поліморфізм та базовий клас Shape для зручного малювання різних фігур. Згідно зі своїм першим варіантом, я реалізувала статичний масив об'єктів, динамічну зміну заголовка вікна та алгоритм побудови прямокутника від центру. Також було налаштовано механіку червоного «гумового сліду» через інверсію кольорів, що дозволяє плавно малювати контур без спотворення інших малюнків на екрані.